



MANIPAL

ACADEMY of HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

CYE 5151 Cryptology Lab – 2

Reg. Number	Name	Branch
261100690010	Hegde Aansh Dilip	Cyber Security

09/09/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Level 1:

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named `cryptology.txt`. Store each statement on a separate line.

Sample statements:

Cryptology is the study of secure communication.

Cryptography focuses on protecting information.

Cryptanalysis focuses on analyzing cryptographic systems.

Plaintext represents the original message.

Ciphertext represents the transformed message.

Cryptanalysis is fun :)

I love Cryptanalysis ;)

```
file = open("cryptology.txt", "w")
```

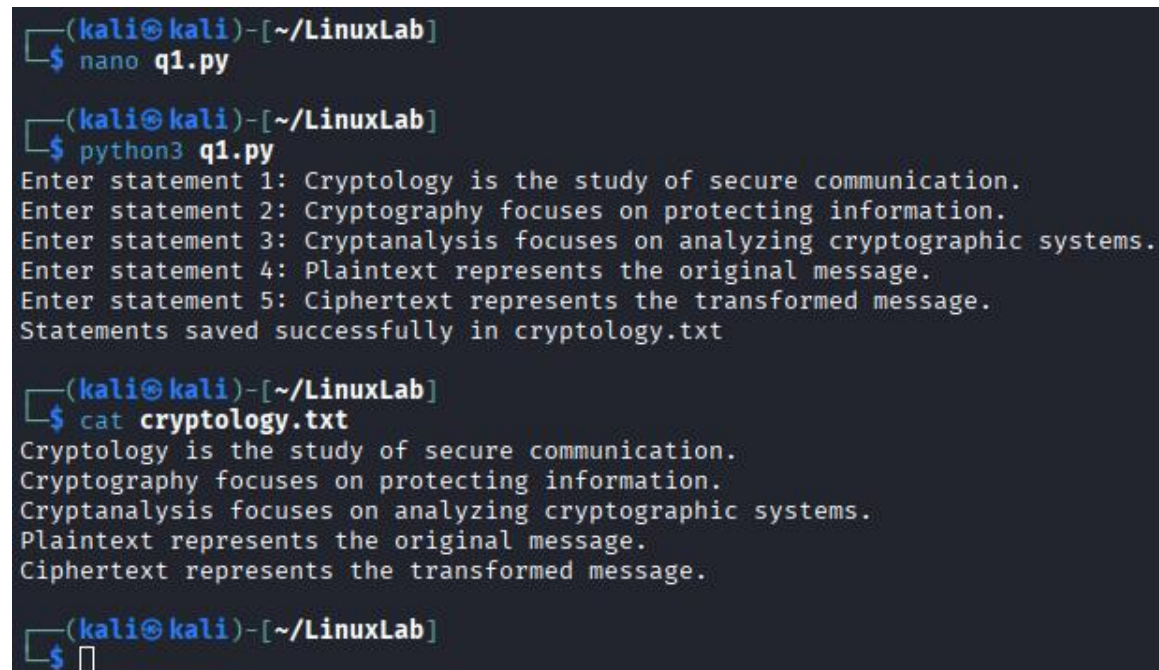
```
for i in range(5):
```

```
    statement = input("Enter statement " + str(i + 1) + ": ")
```

```
    file.write(statement + "\n")
```

```
file.close()
```

```
print("Statements saved successfully in cryptology.txt")
```



```
(kali㉿kali)-[~/LinuxLab]
$ nano q1.py

(kali㉿kali)-[~/LinuxLab]
$ python3 q1.py
Enter statement 1: Cryptology is the study of secure communication.
Enter statement 2: Cryptography focuses on protecting information.
Enter statement 3: Cryptanalysis focuses on analyzing cryptographic systems.
Enter statement 4: Plaintext represents the original message.
Enter statement 5: Ciphertext represents the transformed message.
Statements saved successfully in cryptology.txt

(kali㉿kali)-[~/LinuxLab]
$ cat cryptology.txt
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.

(kali㉿kali)-[~/LinuxLab]
$
```

2. Write a Python program to read cryptology.txt and display its complete contents.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q2.py

(kali㉿kali)-[~/LinuxLab]
$ cat q2.py
file = open("cryptology.txt", "r")

content = file.read()

print(content)

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q2.py
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.

(kali㉿kali)-[~/LinuxLab]
$
```

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q3.py

(kali㉿kali)-[~/LinuxLab]
$ cat q3.py
file = open("cryptology.txt", "r")

line_number = 1

for line in file:
    print(line_number, ":", line.strip())
    line_number = line_number + 1

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q3.py
1 : Cryptology is the study of secure communication.
2 : Cryptography focuses on protecting information.
3 : Cryptanalysis focuses on analyzing cryptographic systems.
4 : Plaintext represents the original message.
5 : Ciphertext represents the transformed message.

(kali㉿kali)-[~/LinuxLab]
$
```

4. write a program to store the output in a file called cryptology_no.txt

```
(kali㉿kali)-[~/LinuxLab]
$ nano q4.py

(kali㉿kali)-[~/LinuxLab]
$ cat q4.py
file = open("cryptology.txt", "r")
output = open("cryptology_no.txt", "w")

line_number = 1

for line in file:
    result = str(line_number) + " : " + line.strip()
    print(result)
    output.write(result + "\n")
    line_number = line_number + 1

file.close()
output.close()

print("Output saved in cryptology_no.txt")

(kali㉿kali)-[~/LinuxLab]
$ python3 q4.py
1 : Cryptology is the study of secure communication.
2 : Cryptography focuses on protecting information.
3 : Cryptanalysis focuses on analyzing cryptographic systems.
4 : Plaintext represents the original message.
5 : Ciphertext represents the transformed message.
Output saved in cryptology_no.txt

(kali㉿kali)-[~/LinuxLab]
$
```

5. Write a program to read cryptology.txt and determine Number of lines, Number of words, Number of characters

```
(kali㉿kali)-[~/LinuxLab]
$ nano q5.py

(kali㉿kali)-[~/LinuxLab]
$ cat q5.py
file = open("cryptology.txt", "r")

lines = 0
words = 0
characters = 0

for line in file:
    lines = lines + 1
    words = words + len(line.split())
    characters = characters + len(line)

file.close()

print("Number of lines:", lines)
print("Number of words:", words)
print("Number of characters:", characters)

(kali㉿kali)-[~/LinuxLab]
$ python3 q5.py
Number of lines: 5
Number of words: 28
Number of characters: 245

(kali㉿kali)-[~/LinuxLab]
$
```

6. Read `cryptology_no.txt` and count the number of: Uppercase letters, Lowercase letters, Digits, Spaces, Special characters

```
(kali㉿kali)-[~/LinuxLab]
$ nano q6.py

(kali㉿kali)-[~/LinuxLab]
$ cat q6.py
file = open("cryptology_no.txt", "r")

uppercase = 0
lowercase = 0
digits = 0
spaces = 0
special = 0

for line in file:
    for ch in line:
        if ch.isupper():
            uppercase = uppercase + 1
        elif ch.islower():
            lowercase = lowercase + 1
        elif ch.isdigit():
            digits = digits + 1
        elif ch == " ":
            spaces = spaces + 1
        elif ch != "\n":
            special = special + 1

file.close()

print("Uppercase letters:", uppercase)
print("Lowercase letters:", lowercase)
print("Digits:", digits)
print("Spaces:", spaces)
print("Special characters:", special)

(kali㉿kali)-[~/LinuxLab]
$ python3 q6.py
Uppercase letters: 5
Lowercase letters: 207
Digits: 5
Spaces: 33
Special characters: 10

(kali㉿kali)-[~/LinuxLab]
$
```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q7.py

(kali㉿kali)-[~/LinuxLab]
$ cat q7.py
word = input("Enter a word to search: ")

file = open("cryptology.txt", "r")

found = False

for line in file:
    if word in line:
        print("Word found:", line.strip())
        found = True

file.close()

if found == False:
    print("Word not found.")

Save using Ctrl + O → Enter → Ctrl + X.

(kali㉿kali)-[~/LinuxLab]
$ nano q7.py

(kali㉿kali)-[~/LinuxLab]
$ cat q7.py
word = input("Enter a word to search: ")

file = open("cryptology.txt", "r")

found = False

for line in file:
    if word in line:
        print("Word found:", line.strip())
        found = True

file.close()

if found == False:
    print("Word not found.")

(kali㉿kali)-[~/LinuxLab]
$ python3 q7.py
Enter a word to search: Cryptology
Word found: Cryptology is the study of secure communication.

(kali㉿kali)-[~/LinuxLab]
```

8. Read `cryptology.txt` and display only the lines containing the word `message`.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q8.py

(kali㉿kali)-[~/LinuxLab]
$ cat q8.py
file = open("cryptology.txt", "r")

for line in file:
    if "message" in line:
        print(line.strip())

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q8.py
Plaintext represents the original message.
Ciphertext represents the transformed message.

(kali㉿kali)-[~/LinuxLab]
$
```

9. Write a Python program to read `cryptology.txt`, convert all alphabetic characters to uppercase, and store the result in a new file called `normalized.txt`.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q9.py

(kali㉿kali)-[~/LinuxLab]
$ cat q9.py
file = open("cryptology.txt", "r")
output = open("normalized.txt", "w")

for line in file:
    output.write(line.upper())

file.close()
output.close()

print("Normalized text saved in normalized.txt")

(kali㉿kali)-[~/LinuxLab]
$ python3 q9.py
Normalized text saved in normalized.txt
```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology_no.txt file.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q10.py

(kali㉿kali)-[~/LinuxLab]
$ cat q10.py
file = open("cryptology_no.txt", "r")

for line in file:
    line = line.strip()

    if line.isalpha():
        print(line)

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q10.py

(kali㉿kali)-[~/LinuxLab]
$
```

11. Write a program to get the string from the user and reverse the string.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q11.py

(kali㉿kali)-[~/LinuxLab]
$ cat q11.py
string = input("Enter a string: ")

reverse = string[::-1]

print("Reversed string:", reverse)

(kali㉿kali)-[~/LinuxLab]
$ python3 q11.py
Enter a string: Cryptology
Reversed string: ygotltpyrC

(kali㉿kali)-[~/LinuxLab]
$
```

12. Improve the above code to reverse the string from the file.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q12.py

(kali㉿kali)-[~/LinuxLab]
$ cat q12.py
file = open("cryptology.txt", "r")

text = file.read()

reverse = text[::-1]

print("Original string:")
print(text)

print("Reversed string:")
print(reverse)

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q12.py
Original string:
Cryptology is the study of secure communication.
Cryptography focuses on protecting information.
Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message.
Ciphertext represents the transformed message.

Reversed string:
.egassem demrofsnart eht stneserper txetrehpiC
.egassem lanigiro eht stneserper txetnialP
.smetsys cihpargotpyrc gnizylana no sesucof sisylanatpyrc
.noitamrofni gnitcetorp no sesucof yhpargotpyrc
.noitacinummoc eruces fo yduts eht si ygotlotpyrc

(kali㉿kali)-[~/LinuxLab]
$
```

13. Simulate the tac command in Python.

```
(kali㉿kali)-[~/LinuxLab]
$ nano q13.py

(kali㉿kali)-[~/LinuxLab]
$ cat q13.py
file = open("cryptology.txt", "r")

lines = file.readlines()

for line in reversed(lines):
    print(line.strip())

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 q13.py
Ciphertext represents the transformed message.
Plaintext represents the original message.
Cryptanalysis focuses on analyzing cryptographic systems.
Cryptography focuses on protecting information.
Cryptology is the study of secure communication.

(kali㉿kali)-[~/LinuxLab]
$
```

Level 2: Regex

security_data.txt:

User alice logged in from 192.168.10.15

User bob logged in from 10.10.5.21

Contact admin@securitylab.com

Failed login attempts: 5

Session ID: SEC-2026-1045

Hash: a94f3c2d8e71b05f

Port 443 connection established

User admin failed authentication

1. Write a Python program to read security_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```
(kali㉿kali)-[~/LinuxLab]
$ nano security_data.txt

(kali㉿kali)-[~/LinuxLab]
$ nano regex1.py

(kali㉿kali)-[~/LinuxLab]
$ cat regex1.py
import re

file = open("security_data.txt", "r")
text = file.read()

pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b'

ips = re.findall(pattern, text)

print("IPv4 addresses found:")

for ip in ips:
    print(ip)

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 regex1.py
IPv4 addresses found:
192.168.10.15
10.10.5.21

(kali㉿kali)-[~/LinuxLab]
$
```

2. Write a Python program to identify and display all email-address-like patterns present in security_data.txt.

```
(kali㉿kali)-[~/LinuxLab]
$ nano regex2.py

(kali㉿kali)-[~/LinuxLab]
$ cat regex2.py
import re

file = open("security_data.txt", "r")
text = file.read()

pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b'

emails = re.findall(pattern, text)

print("Email addresses found:")

for email in emails:
    print(email)

file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 regex2.py
Email addresses found:
admin@securitylab.com
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

```
(kali㉿kali)-[~/LinuxLab]
$ nano regex3.py

(kali㉿kali)-[~/LinuxLab]
$ cat regex3.py
import re
file = open("security_data.txt", "r")
text = file.read()
session_ids = re.findall(r'SEC-\d{4}-\d{4}', text)
print("Session IDs:")
for session in session_ids:
    print(session)
file.close()

(kali㉿kali)-[~/LinuxLab]
$ python3 regex3.py
Session IDs:
SEC-2026-1045

(kali㉿kali)-[~/LinuxLab]
$
```

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```
(kali@kali)-[~/LinuxLab]
$ nano regex4.py

(kali@kali)-[~/LinuxLab]
$ cat regex4.py
import re

file = open("security_data.txt", "r")
text = file.read()

pattern = r'\b[0-9a-fA-F]+\b'

hex_values = re.findall(pattern, text)

print("Hexadecimal sequences found:")

for value in hex_values:
    print(value)

file.close()

(kali@kali)-[~/LinuxLab]
$ python3 regex4.py
Hexadecimal sequences found:
192
168
10
15
10
10
5
21
5
2026
1045
a94f3c2d8e71b05f
443
```

5. Write a Python program to read security_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
(kali@kali)-[~/LinuxLab]
$ nano regex5.py

(kali@kali)-[~/LinuxLab]
$ cat regex5.py
import re

file = open("security_data.txt", "r")

for line in file:
    if re.search(r'\bfailed\b', line, re.IGNORECASE):
        print(line.strip())

file.close()

(kali@kali)-[~/LinuxLab]
$ python3 regex5.py
Failed login attempts: 5
User admin failed authentication

(kali@kali)-[~/LinuxLab]
$
```

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```
(kali@kali)-[~/LinuxLab]
$ nano regex6.py

(kali@kali)-[~/LinuxLab]
$ cat regex6.py
import re

file = open("security_data.txt", "r")

for line in file:
    match = re.search(r'\bPort\s+(\d+)\s+connection\s+established\b', line)

    if match:
        print("Port number:", match.group(1))

file.close()

(kali@kali)-[~/LinuxLab]
$ python3 regex6.py
Port number: 443

(kali@kali)-[~/LinuxLab]
```